

Trabajo Autonomo N.º 1

Estudiante:

Angel Rafael Parraga Zambrano

Materia:

Logica de Programacion

Docente:

Vizcaino Imacaña Fernanda Paulina

Fecha:

28 de mayo de 2026

1. Introducción

En el contexto actual de la seguridad informática, la gestión de contraseñas representa uno de los desafíos más frecuentes tanto para usuarios comunes como para profesionales del área tecnológica. El uso de contraseñas débiles o reutilizadas es una de las principales causas de brechas de seguridad a nivel global.

El presente documento describe el proceso de diseño previo al desarrollo de un **Generador Seguro de Contraseñas**, un programa de software cuyo propósito es asistir al usuario en la creación de contraseñas que cumplan con criterios robustos de seguridad, de forma rápida y sin necesidad de conocimientos técnicos especializados.

El diseño se aborda mediante dos tipos de diagramas:

- **Diagramas funcionales:** representan las funcionalidades del sistema y la interacción del usuario (diagrama de casos de uso y diagrama de flujo).
- **Diagrama de arquitectura:** muestra la estructura interna del sistema y la relación entre sus componentes principales.

Este documento constituye la fase de análisis y diseño, previa a cualquier implementación de código.

2. Selección del Programa

De entre las opciones disponibles para el desarrollo, se seleccionó el programa **Generador Seguro de Contraseñas** por las siguientes razones:

1. **Relevancia práctica:** es una herramienta de utilidad real e inmediata para cualquier usuario con acceso a sistemas digitales.
2. **Aplicación de conceptos de seguridad:** permite trabajar con conceptos de entropía, aleatoriedad criptográfica y política de contraseñas.
3. **Complejidad funcional adecuada:** el programa requiere manejo de lógica de negocio, interfaz de usuario e interacción con el sistema operativo (portapapeles, almacenamiento), lo que lo hace un buen ejercicio de diseño.
4. **Escalabilidad:** el diseño puede extenderse fácilmente con funcionalidades adicionales como gestión de historial, exportación o integración con un gestor de contraseñas.

Descripción general del programa

El *Generador Seguro de Contraseñas* es una aplicación que permite al usuario configurar los parámetros de una contraseña (longitud y tipos de caracteres permitidos), generarla mediante un algoritmo criptográficamente seguro, evaluar su fortaleza y copiarla al portapapeles. Adicionalmente, el sistema mantiene un historial de contraseñas generadas durante la sesión.

3. Análisis del Problema

3.1. Definición del problema

Los usuarios frecuentemente crean contraseñas débiles, predecibles o reutilizan las mismas credenciales en múltiples servicios. Esto los expone a ataques de fuerza bruta, diccionario o de relleno de credenciales (*credential stuffing*). La solución es contar con una herramienta que genere contraseñas fuertes de forma automatizada.

3.2. Requerimientos funcionales

ID	Nombre	Descripción
RF-01	Configurar longitud	El usuario puede definir la longitud de la contraseña (mínimo 8, máximo 128 caracteres).
RF-02	Seleccionar caracteres	El usuario elige qué conjuntos de caracteres incluir: mayúsculas, minúsculas, dígitos y símbolos especiales.
RF-03	Generar contraseña	El sistema genera una contraseña aleatoria conforme a los parámetros configurados.
RF-04	Evaluar fortaleza	El sistema calcula y muestra un indicador visual de la fortaleza de la contraseña.
RF-05	Copiar al portapapeles	El usuario puede copiar la contraseña con un clic.
RF-06	Ver historial	El sistema almacena y muestra las contraseñas generadas en la sesión.
RF-07	Limpiar historial	El usuario puede borrar el historial de la sesión actual.

Cuadro 1: Requerimientos funcionales del sistema

3.3. Requerimientos no funcionales

- **Seguridad:** la aleatoriedad debe provenir de un generador criptográficamente seguro (CSPRNG), no de funciones pseudoaleatorias simples.
- **Usabilidad:** la interfaz debe ser intuitiva, sin requerir conocimientos técnicos previos.
- **Portabilidad:** el programa debe ejecutarse en los sistemas operativos más comunes (Windows, macOS, Linux).
- **Rendimiento:** la generación de la contraseña debe completarse en menos de un segundo.

3.4. Actores del sistema

El único actor externo identificado es el **Usuario**, quien interactúa directamente con la interfaz del programa para configurar los parámetros, solicitar la generación y usar la contraseña resultante.

4. Diagramas Funcionales

Los diagramas funcionales describen *qué* hace el sistema y *cómo* interactúa el usuario con él. Se presentan dos tipos: el diagrama de casos de uso (visión estática de funcionalidades) y el diagrama de flujo (visión dinámica del proceso principal).

4.1. Diagrama de Casos de Uso

El diagrama de casos de uso muestra las interacciones posibles entre el actor (*Usuario*) y el sistema. Cada caso de uso representa una funcionalidad observable desde el exterior del sistema.

Se identifican las siguientes relaciones:

- Los casos de uso *Configurar longitud* y *Seleccionar tipos de caracteres* tienen una relación «include» con *Generar contraseña*, ya que siempre se ejecutan como parte previa a la generación.
- *Generar contraseña* incluye implícitamente *Evaluar fortaleza*, ya que la evaluación ocurre automáticamente tras cada generación.

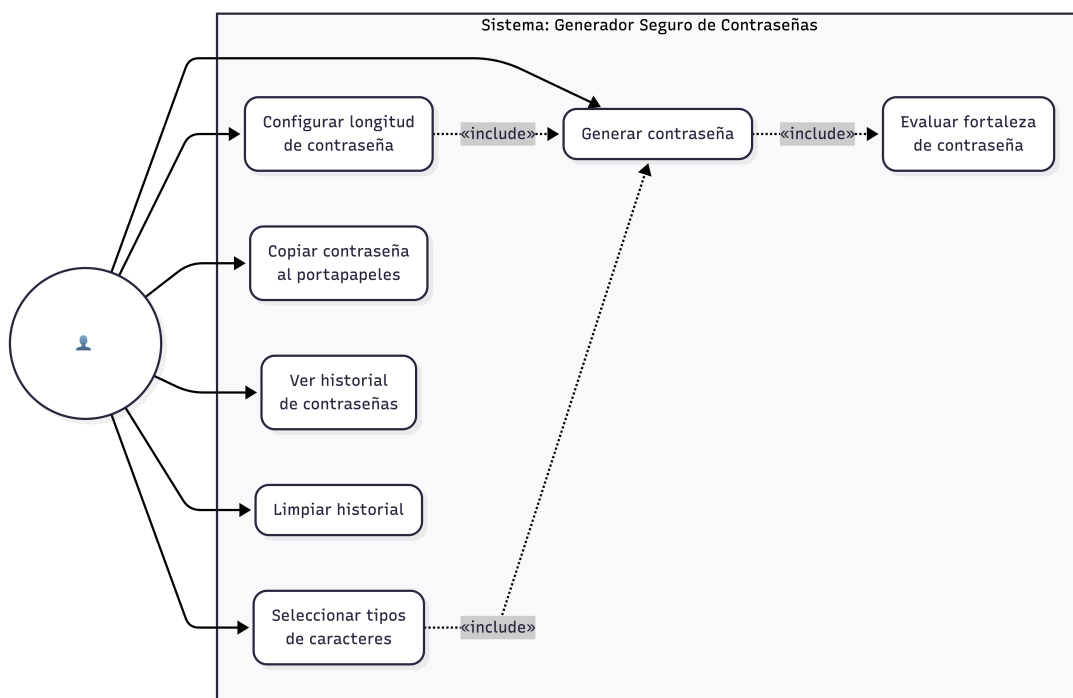


Figura 1: Diagrama de Casos de Uso — Generador Seguro de Contraseñas

4.2. Diagrama de Flujo del Proceso

El diagrama de flujo representa la secuencia lógica de pasos que ejecuta el sistema desde que el usuario configura los parámetros hasta que finaliza la interacción. Incluye las deci-

siones de validación y las diferentes acciones que el usuario puede tomar una vez generada la contraseña.

Pasos principales del flujo:

1. El usuario define los parámetros (longitud y tipos de caracteres).
2. El sistema valida que los parámetros sean correctos.
3. Se construye el conjunto de caracteres disponibles según la configuración.
4. Se seleccionan caracteres aleatoriamente mediante un CSPRNG.
5. Se evalúa la fortaleza; si es insuficiente, se regenera.
6. Se muestra la contraseña al usuario.
7. El usuario decide copiar, guardar en historial, generar una nueva o salir.

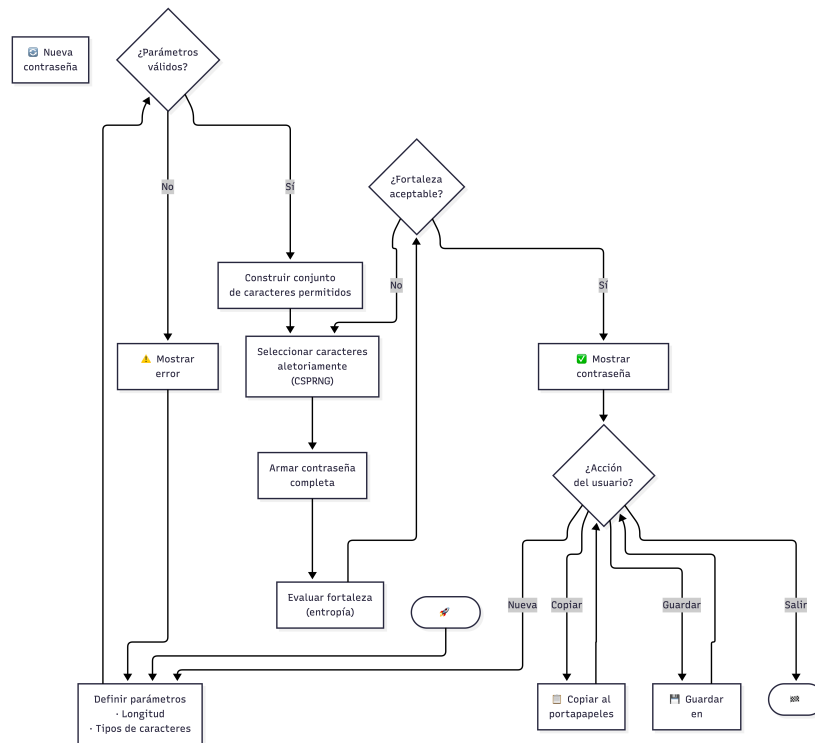


Figura 2: Diagrama de Flujo — Proceso de Generación de Contraseña

5. Diagrama de Arquitectura

El diagrama de arquitectura describe la estructura interna del sistema, mostrando sus componentes principales y las dependencias entre ellos. Se adoptó una arquitectura en **tres capas** (*three-tier architecture*), separando claramente la presentación, la lógica de negocio y el acceso a datos.

5.1. Capas del sistema

Capa de Presentación (UI): gestiona todo lo que el usuario ve e interactúa directamente. Contiene el panel de configuración, el visualizador de la contraseña generada, el indicador de fortaleza y el panel de historial.

Capa de Lógica de Negocio: concentra las reglas del sistema. Incluye el *Motor de Generación* (que produce la contraseña usando CSPRNG), el *Evaluador de Fortaleza* (que calcula la entropía y clasifica la contraseña) y el *Gestor de Configuración* (que valida y administra los parámetros del usuario).

Capa de Datos: administra la persistencia y los recursos estáticos. Comprende el *Almacenamiento Local* (historial de sesión), la *Configuración de Usuario* (preferencias guardadas) y la *Biblioteca de Caracteres* (conjuntos de caracteres disponibles para la generación).

5.2. Flujo de comunicación entre capas

La comunicación fluye de forma unidireccional descendente: la UI invoca servicios de la capa de lógica, y ésta accede a la capa de datos cuando lo requiere. Ninguna capa accede directamente a una capa no adyacente, lo que garantiza bajo acoplamiento y alta cohesión.

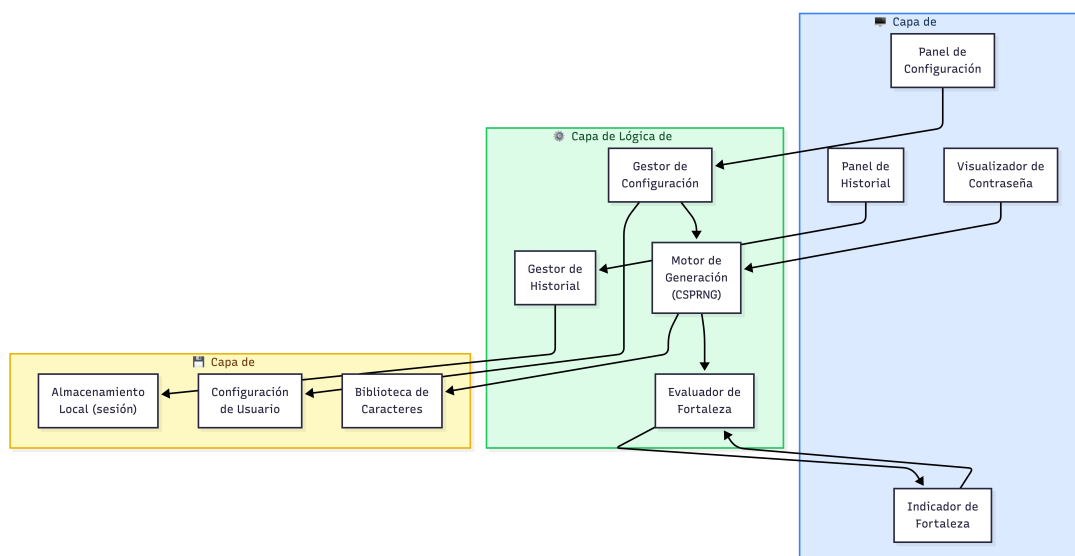


Figura 3: Diagrama de Arquitectura del Sistema (3 capas)

6. Conclusiones

El ejercicio de diseño previo al desarrollo permitió comprender con mayor claridad el alcance y la estructura del **Generador Seguro de Contraseñas** antes de escribir una sola línea de código. Las conclusiones principales son:

1. **La fase de diseño es esencial:** identificar los requerimientos y plasmarlos en diagramas evita ambigüedades durante el desarrollo y reduce el retrabajo.
2. **La separación por capas mejora la mantenibilidad:** la arquitectura de tres capas permite modificar, por ejemplo, la interfaz gráfica sin alterar la lógica de generación, y viceversa.
3. **Los casos de uso centran el diseño en el usuario:** el diagrama de casos de uso dejó en evidencia que el usuario es el único actor y que todas las funcionalidades deben estar orientadas a facilitar su experiencia.
4. **El diagrama de flujo revela decisiones críticas:** pasos como la validación de parámetros y la evaluación de fortaleza, que podrían pasarse por alto, quedaron explícitos gracias al diagrama.
5. **La seguridad debe considerarse desde el diseño:** la elección de un CSPRNG en lugar de un generador pseudoaleatorio simple es una decisión de arquitectura, no de implementación tardía.

Con estos diagramas como guía, el desarrollo del programa contará con una base sólida que asegura coherencia entre el diseño y el producto final.